An-Najah National University
Faculty of Engineering and IT

جامعة النجاح الوطنية
كلية الهندسة وتكنولوجيا المعلومات

Computer Engineering Department
Dos (10636456)
Project p1

Instructor Name: Dr.Samer Alarandi

Students Name: Ali Odeh (12112361) & Abd Albaset Rajab (12042362)

Academic Year: Spring 2026

Bazar.com: A Multi-tier Online Book Store

Design Document

1. Overview

Bazar.com is a minimal two-tier, microservices-based online bookstore. It sells exactly four books and is built to demonstrate a distributed multi-tier web architecture. The system consists of three independent microservices that communicate over HTTP REST. We using Node.js with the Express framework, which provides a lightweight and efficient way to build RESTful microservices.

- Frontend Service (port 3000) — accepts client requests and routes them to backend services
- Catalog Service (port 3001) — manages book data (inventory, prices, topics)
- Order Service (port 3002) — processes purchases and records order history

All three services are containerized using Docker and orchestrated with Docker Compose, allowing the system to run in a fully distributed manner across three isolated containers.

2. System Architecture

2.1 High-Level Design

The system follows a two-tier architecture:

- Tier 1 (Frontend): A single Express.js microservice that acts as the API gateway. It exposes three REST endpoints to clients and proxies requests to the appropriate backend service.
- Tier 2 (Backend): Two microservices — the Catalog Service and Order Service — that handle data storage and business logic. They communicate with each other as needed (e.g., Order queries Catalog to check stock before purchasing).

2.2 Service Responsibilities

Service	Endpoints	Responsibility
Frontend (3000)	GET /search/:topic GET /info/:id POST /purchase/:id	API gateway; proxies to catalog or order
Catalog (3001)	GET /search/:topic GET /info/:id GET /query/item/:id GET /query/topic/:topic PUT /update/:id	Reads/writes catalog.json; handles book queries and stock updates
Order (3002)	POST /purchase/:id GET /orders	Verifies stock, decrements inventory, records order to orders.json

2.3 Inter-Service Communication

Services communicate exclusively via HTTP REST using the axios library. Docker Compose's internal DNS allows services to refer to each other by name (e.g., http://catalog:3001). The communication flow is:

- search / info: Client → Frontend → Catalog → response back to client
- purchase: Client → Frontend → Order → Catalog (stock check + decrement) → response back to client

3. How It Works

3.1 Search

A client sends GET /search/:topic to the frontend. The frontend proxies this to the catalog service at GET /search/:topic. The catalog reads catalog.json, filters books by topic (case-insensitive), and returns a list of matching books with their id and title.

3.2 Info

A client sends GET /info/:id to the frontend. The frontend proxies this to GET /info/:id on the catalog service. The catalog looks up the book by ID and returns its title, quantity, and price.

3.3 Purchase

A client sends POST /purchase/:id to the frontend. The frontend forwards this to the order service at POST /purchase/:id. The order service then:

- Queries GET /query/item/:id on the catalog to get full book details
- Checks that quantity > 0; returns 400 error if out of stock
- Sends PUT /update/:id to the catalog with { quantityChange: -1 } to decrement stock
- Appends a new order record to orders.json with orderId, itemId, title, price, and timestamp
- Returns the completed order object to the client

3.4 Data Persistence

Both the catalog and order data are stored as JSON files on disk (catalog.json and orders.json in the shared /data directory). Both files are mounted into the catalog and order containers via a Docker volume, so data persists across container restarts.

The system supports concurrent requests using Node.js's asynchronous event-driven architecture and Express's built-in request handling.

Additionally, the system logs all incoming requests and important events (such as successful purchases) to the console to demonstrate system behavior and facilitate debugging.

4. Design Tradeoffs

4.1 JSON Files vs. Database

We chose plain JSON files over a database (even SQLite) for simplicity and ease of inspection. This is appropriate for a four-book store with low traffic. The tradeoff is that file I/O is not atomic — concurrent writes could corrupt data under heavy load. For this lab's scale, the risk is acceptable.

4.2 Synchronous REST vs. Message Queues

All inter-service calls are synchronous HTTP requests. This is simple and easy to debug, but means a slow catalog service will block an order request. An asynchronous message queue (e.g., RabbitMQ) would improve resilience but adds significant complexity not warranted for this scale.

4.3 No Caching

Every request reads from disk. Adding an in-memory cache on the frontend (e.g., a simple JavaScript Map with a TTL) would dramatically reduce catalog reads, but introduces cache invalidation complexity. Given the small dataset, caching was omitted.

4.4 Docker Compose vs. Distributed Machines

Docker Compose runs all three containers on a single host, simulating distribution. The service URLs (http://catalog:3001, http://order:3002) use Docker's internal DNS. Replacing these with real IPs or hostnames would allow true cross-machine deployment with no code changes.

5. Possible Improvements and Extensions

- Caching layer: Add a simple in-memory LRU cache on the frontend to avoid repeated catalog reads for popular books.
- Database: Replace JSON files with SQLite for atomic transactions, preventing data corruption under concurrent purchases.
- Authentication: Add JWT-based auth to protect the purchase endpoint.
- Stock replenishment endpoint: Add a PUT /restock/:id endpoint on the catalog to simulate periodic stock arrivals as described in the lab spec.
- Health checks: Add /health endpoints to each service and configure Docker Compose healthchecks so dependent services wait until dependencies are truly ready.

- Horizontal scaling: Run multiple catalog replicas behind a load balancer (e.g., Nginx) for higher throughput.
- Order history endpoint: Expose GET /orders on the frontend (currently only accessible directly on the order service).

6. Known Limitations

- Race condition on concurrent purchases: If two clients purchase the last copy of a book simultaneously, both reads may see quantity=1 before either write completes, resulting in quantity going negative. A mutex or atomic compare-and-swap would fix this.
- No input validation on price updates: The PUT /update/:id endpoint accepts any price value without bounds checking.
- Order ID generation is not thread-safe: generateOrderId reads the last entry and increments it. Concurrent writes could produce duplicate IDs.

7. How to Run

7.1 Prerequisites

- Docker and Docker Compose installed
- Port 3000 available on the host machine

7.2 Start the System

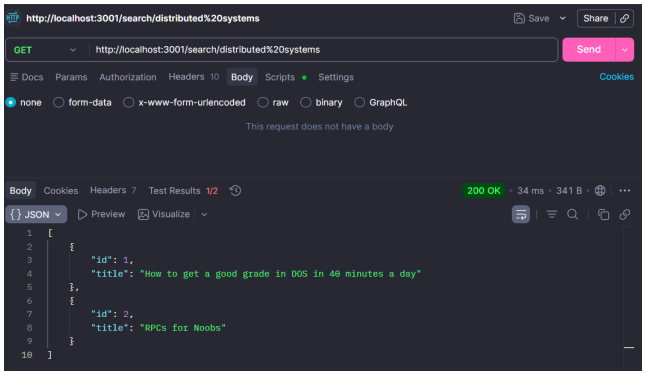
```
git clone <your-repo-url>
cd bazar-com
docker-compose up --build
```

This builds and starts all three services. The frontend is accessible at http://localhost:3000.

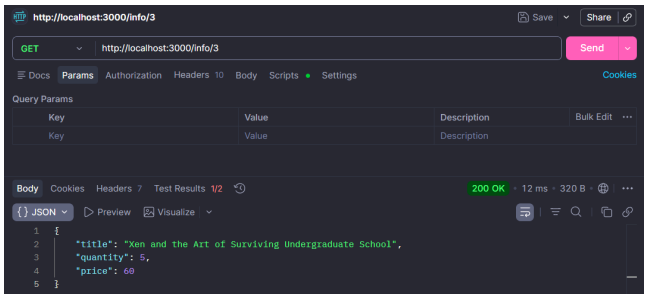
7.3 Example Requests

Note: API requests were tested using **Postman** and **Google Chrome** to verify functionality and observe responses.

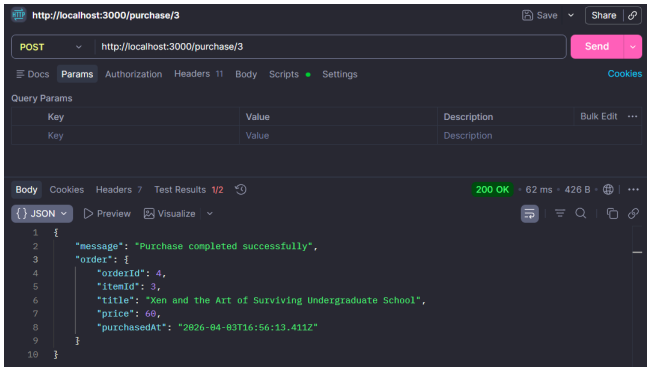
Search for books by topic:



Get info about a specific book:



Purchase a book:



Note: All test cases and output results for the project are included in the **/docs** folder in the GitHub repository.

Github Repo:

https://github.com/Ali-Odeh/Dos_P1-Bazar.com-A-Multi-tier-Online-Book-Store.git